Worth the Lookup: Budgeted Relational Retrieval for Call-List Prioritization

Max Rodriguez **Stanford University** maxrod@stanford.edu

Project type: real-world application of GNNs (blog post + Colab). **Group:** Max Rodriguez (solo). Not shared with another course.
Code: `https://github.com/MaxLuisRodriguez/CS224W-fraud-detection-machine-learning-with-graphs/tree/milestone` (main branch up to date with commit used for this report, tagged `milestone`; see README for code map and run commands).

1 Problem and motivation

With modern parallel dialing capabilities, a rep can make upwards of hundreds of calls a day. Before each batch of calls, a sales rep's AI research agent can look up a prospect's recent activity, the accounts and events it touches, along with other buying signals. Every lookup carries a cost, whether that is enrichment credits, LLM tokens, or latency before the dialer connects. No team has the capacity to research every prospect in depth, so the real decision ends up being *which lookups to make for the entire prospective call list* within a budget. In this domain, sales data are linked tables, and the prospects on one list are often connected to the same accounts and events (Figure 1). That couples the prospects in two ways.

Shared information. Fetching one shared event informs every prospect connected to it, so its value can be aggregated as the sum of its effects on all of them.

A shared decision. A rep calls only the top K candidates of a call list, hence a lookup only really matters if it can alter who lands in the top K, which depends on everyone other prospect's scores. Consider the following scenario: five prospects score 0.80, 0.55, 0.53, 0.20 and 0.10, and the rep gets to call two. A lookup that moves the fourth prospect from 0.20 to 0.30 improves its prediction but does not change the top K call list. On the other hand, a lookup that moves the third prospect from 0.53 to 0.60 swaps who gets called.

Thus, we have three objectives/goals we can optimize for, and that are easy to conflate: making each prediction more accurate (minimizing cross-entropy loss), ranking the whole list better (AUROC, average precision), and choosing the right top K (Precision@K) candidates. Milestone results show that these can diverge depending on how policies attribute value to lookups.

Research question. Given a call list of N prospects and a budget of B lookups, which lookups should an agent make so that a graph neural network (GNN) ranks the list best (in accordance to our objectives defined above), and in particular picks the right top K?

Contributions. My milestone consists of three core parts, and details a fourth to be executed after the milestone. (i) Development of a relational GNN that scores a prospect when only part of its neighborhood has been retrieved, allowing the model to score prospects at different budgets. (ii) Creation of a lookup environment that attributes cost to retrieval. It has two actions, *pull* and *enrich*, each costing one unit of budget, and a cache shared across the list, so any shared entity is only paid for once. (iii) A comparison of lookup policies by their *budget frontier*, ranking quality against lookups per prospect, which shows how well each policy allocates a fixed budget. (iv) After the milestone, I will implement a learned policy that predicts the value of each lookup for the whole list.

Re-defined scope after mentor feedback. I complete a comparison of lookup policies on one dataset before expanding scope, as learning the lookup value adds significant complexity. Every result below is therefore on `rel-event`. `rel-f1` is used only to develop and test code, and `rel-avito` is a post-milestone stretch goal. Two hypotheses from my proposal have been re-scoped: **(H1)** frontiers

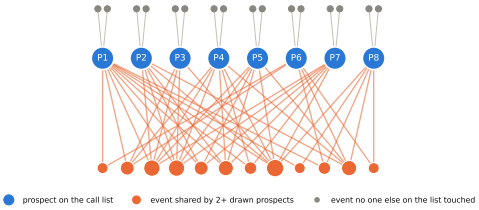


Figure 1: A real call list example (`rel-event`, 2012-11-21): random 128-prospect list, with the 8 prospects sharing the most events (blue), their 12 most-shared events (orange; larger = shared by more), and two unshared events each (grey). One orange lookup informs several prospects.

Table (node type)	Nodes	Foreign keys (edges)
`users`	37,143	–
`events`	2,459,811	creator (945)
`event_attendees`	8,430,002	event (8.43M), user (49.8K)
`event_interest`	14,978	event (14.1K), user (15.0K)
`user_friends`	30,386,403	user (30.0M), friend (215K)

Table 1: The `rel-event` graph: 41.3M nodes (rows up to the test cutoff), 5 node types, 6 foreign keys yielding 12 edge types. An edge exists only where the key points to a row in the database; here, most attendance rows link to an event but not to a known user. `user_friends` has no timestamp, so friendships are always visible.

are concave: with an amortized quarter-of-a-lookup per prospect, a policy recovers most of the AUROC gap between zero lookups and full information; **(H2)** a model which is trained on randomly hidden rows will score higher (AUROC and Precision@13) than one trained on full neighborhoods when little is revealed during validation/test-time; **(H3)** retrieval value is concentrated: as measured through AUROC gain per lookup when one table is pulled for every prospect, lookups from one or two tables will account for most of the gain; **(H4)** the AUROC advantage of a batch-aware policy over a per-prospect policy grows with a list's shared lookups, defined further in Section 2; **(H5)** a learned batch value beats the best heuristic (to be tested after the milestone).

Status. For this milestone, everything ran on `rel-event`: the data pipeline, the partially observed GNN, the lookup environment, the five policy heuristics, and the frontier runner. There were three main findings from my milestone work. (1) With just a quarter of a lookup per prospect, the best policy heuristic recovers 83% of the AUROC gap between zero lookups and full information (supporting H1). (2) Training on randomly hidden rows helps much less than anticipated (working against H2). (3) One table, for the prospects' own invitation responses, carries most of the retrieval value (in favor of H3). This helps to explain why researching every prospect until every one has some evidence beats a heuristic that prioritizes enrichment of shared events (H4). Additionally, I found and fixed an important leak: a timestamp on the target's own row (`joinedAt`) can postdate the prediction time. Every `rel-event` model was retrained without it, and all results below use the fixed models; notably, the fix weakened the LightGBM baseline and strengthened the full-trained GNN (Section 4.1).

2 Data and processing

Dataset and task. I use the `user-ignore` task of `rel-event` from RelBench [6], which is a public benchmark of real relational databases with temporal splits. The database comes from the Kaggle Event Recommendation Engine Challenge, and was downloaded from Hugging Face. In `rel-event`, users receive invitations to events and respond yes, maybe or no, or can never respond. Prediction labels are kept for users at a time t, and record whether the user will ignore more than two invitations in the seven days after t (here, ignoring means never responding). The label comes from attendance records after t, and the model only observes records dated at or before t. Training, validation, and test splits are by time. Training has 19,239 labeled users over 22 prediction dates (16.9% positive). Validation is one day, 2012-11-21, with 2,013 users (11.1% positive): a single day's call list, from which lists of 128 prospects are drawn. The test day (1,958 users) is currently held out and will be used once for the final blog results.

So how can this stand in for call-list scenarios? `rel-event` is a publicly available proxy, not sales data. Users represent prospects, events represent the accounts and interactions they share, and the label represents an engagement outcome used to rank a list. The underlying structure of linked tables, entities shared across a list, a fixed lookup budget, and a top-K decision also appears in sales scenarios where real CRM and call data are available. I do not claim that the specific results here transfer directly to a CRM.

Building the heterogeneous graph. Following relational deep learning (RDL) [2], every row becomes a node typed by its table, and every primary–foreign-key link becomes an edge in both directions (`f2p_<key>` and its inverse, `rev_f2p_<key>`). This yields 5 node types and 12 edge types

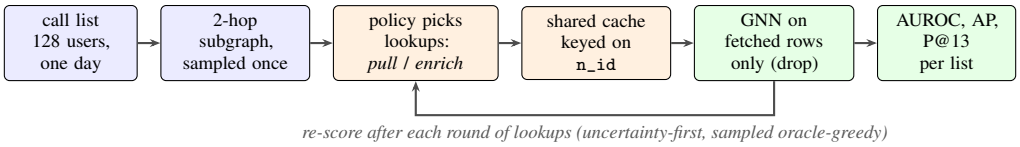


Figure 2: The evaluation pipeline. A call list is sampled once. A policy spends the budget in rounds of $N/4$ paid lookups; an action whose rows are already cached costs nothing. The trained GNN scores the list from the fetched rows, and the metrics are recorded each time the spend reaches a budget.

(Table 1). For example, the attendance row (user 17, event 812, status *invited*, dated 2012-11-14) becomes an `event_attendees` node with two features, its status and date, and with edges to user 17 and event 812 in both directions. RelBench proposes a semantic type for every column, which determines its encoding. Columns containing numerical values are normalized. Categorical columns receive learned embeddings. Text columns, such as a user's location, are embedded with 300-d GloVe word vectors [5]. RelBench's `make_pkey_fkey_graph` function performs this conversion, turning each table into PyTorch Frame tensors; `events`, for instance, becomes a $2.46\text{M} \times 108$ frame.

Temporal sampling. To score a user at time t, PyG's temporal `NeighborLoader` [1] samples up to 128 neighbors per edge type at hop 1 and then 64 neighbors at hop 2, only among rows dated at or before t; the target's own row is always included, regardless of its date (Section 4.1). This sampling is disjoint: explicitly, each target gets its own tree, because two targets can have different prediction times and, thus, may see different parts of the same neighborhood. As a result, an event that two prospects both attended appears twice in a batch, as two separate nodes. The model treats the copies as unrelated, which works fine for prediction but not retrieval. For retrieval, an agent that fetches the event once should see it under both prospects who attended. PyG retains each sampled node's original row id (`n_id`), so the two copies maintain a shared id, and our lookup cache is then keyed on that id (Section 3.3).

Call lists. A call list consists of $N = 128$ users drawn at random (seeded) from the validation day. The users' neighborhoods are sampled once into a single batch, with every policy acting on that same subgraph. Table 2 shows one example list (seed 0). On it, every prospect has friendship rows, but only 86 of 128 have attendance rows, 15 have interest clicks and 10 created events, so the list offers 239 pulls (about 1.9 per prospect). I measure a list's *sharing* per entity type as

$$\text{sharing} = \frac{|\{\text{entities listed by two or more prospects}\}|}{|\{\text{entities listed by any prospect}\}|}. \tag{1}$$

For this particular list, 930 distinct events lie behind the attendance rows, and 33% of them are listed by two or more prospects. Of the 1,296 friends who are known users, 11% are shared. The difference between sampled copies and distinct rows matters for cost; more explicitly: a lookup pays for a distinct row once, independent of how many copies the sampler made. Fetching everything costs 2,752 paid lookups per list on average (2,287min–3,509max lookups over 100 lists), which averages out to about 21.5 per prospect.

Table	Sampled rows	Distinct	Role in the lookup environment
`user_friends`	17,630	17,628	hop 1 for all 128 prospects (one pull each)
`event_attendees`	1,647	1,647	hop 1 for 86 prospects (1,519 rows); 128 rows of events prospects created
`event_interest`	116	116	hop 1 for 15 prospects
`events`	1,672	1,060	hop 2: 930 events behind attendance rows (33% listed by 2+ prospects); hop 1: 49 created by 10 prospects
`users`	1,477	1,303	the 128 prospects, and 1,296 friends who are known users (11% listed by 2+)

Table 2: One 128-prospect call list on `rel-event` (list seed 0). Sampled rows count every copy the per-target sampler made; distinct rows count each database row once.

3 Model design and metrics

3.1 Model

I use RelBench's RDL baseline GNN [2], built in PyG [1]. Each node v has a table type $T(v)$, features x_v and a timestamp $\tau(v)$. To score a prospect at time t, each row starts as $h_v^{(0)} = \mathrm{Enc}_{T(v)}(x_v) + \phi(t - \tau(v))$; the encoding ϕ of the row's age at t is omitted for the prospect's own row (Section 4.1). Two heterogeneous GraphSAGE layers [3] then update every node:

$$h_v^{(\ell+1)} = \mathrm{ReLU}\Big(\mathrm{LN}\Big(\textstyle\sum_{r\in\mathcal{R}(v)}\big[W_{r,1}^{(\ell)} h_v^{(\ell)} + W_{r,2}^{(\ell)} \sum_{u\in\mathcal{N}_r(v)} h_u^{(\ell)}\big]\Big)\Big). \tag{2}$$

Here, a relation r is a (source type, edge type, target type) triple, and $\mathcal{R}(v)$ holds the relations ending at v's type. $\mathcal{N}_r(v)$ is the set of v's sampled neighbors under r. $W_{r,2}$ transforms the summed neighbor states and $W_{r,1}$ transforms v's own state; consequently, rows like attendance rows and friendship rows send differently weighted messages. Layer normalization (LN) rescales each node's vector, keeping the layer stable even when nodes carry many neighbors. Neighbor states are summed rather than averaged to keep counts, such as how many invitations went unanswered, which a mean or a max discards. GIN [9] shows that a sum followed by an MLP can be injective on multisets of countable features; our layer applies only a linear map after the sum, allowing it to keep count information; however, it loses guarantee of injectivity.

Additional model specs. The model has 5.92M parameters. Training minimizes binary cross-entropy with Adam (learning rate 0.005, batch 512, 20 epochs), as in RelBench [6].

3.2 Partial retrieval (my extension)

At prediction time, the agent sees only the set of observations O, the rows fetched so far; as mentioned previously, the prospects' own rows are always included O. I experiment with two ways of hiding the unfetched rows (Table 3). In *mask* mode an unfetched row's input is replaced by a learned per-table "stub" vector:

$$\tilde{h}_v^{(0)} = m_v\, h_v^{(0)} + (1 - m_v)\, e_{T(v)}^{\mathrm{mask}}, \qquad m_v = \mathbb{1}[v \in O]. \tag{3}$$

Revelation is keyed on `n_id`, so fetching a row reveals all its sampled copies at once. I train two scoring regimes. *Full-trained* uses the standard method, and is trained with full, unhidden information. *Random-trained*, on the other hand, samples a $p \sim U(0, 1)$ per batch and reveals each distinct row with probability p. All lookup experiments use drop mode, the mode both regimes were trained in. Mask mode needs its own trained models and is a planned ablation.

	Drop	Mask
What the model knows about an unfetched row	Nothing: not its existence, type or edges	That it exists, its table and its edges; its content is a learned placeholder
Effect on a sum in Eq. 2	One fewer term	Same number of terms, placeholder values
Trade-off	Simple; cannot tell "unread" from "absent"	Keeps structure; needs models trained with placeholders

Table 3: Two ways to hide rows that have not been fetched. All lookup experiments use drop mode.

3.3 Lookup environment

Two actions retrieve rows:

- `pull`(i, T) reveals all of prospect i's sampled rows in table T (up to 128 per relation). On `rel-event` these are attendance records, friendships in either direction, interest clicks, or events the user created. A pull also *lists* the entities to which those rows point: ids become known, but contents (which require enrichment) do not.
- `enrich`(T, v) reveals exactly one listed row v, in every sampled copy: an attended event, a friend who is a known user, or an attendance row of an event the user created.

Notably, an entity can be enriched only after a pulled row lists it, as with real CRM and enrichment APIs. An action that reveals at least one new row costs one *paid lookup*. An action whose rows are all cached costs nothing, and each pull only needs to be made once. Overall, a budget of b lookups per prospect implies bN paid lookups for a list. *Full retrieval*, then, executes every pull and continues to enrich until nothing new is listed; its paid cost is the list's *full cost*.

Example. `pull`(Ana, attendance) and `pull`(Ben, attendance) cost one unit each, and both list event 812. `enrich`(events, 812) costs one and reveals event 812 under both prospects. Thus, a second enrich of event 812 would cost nothing.

The objective and the batch value. For a list $Q = \{q_1, \ldots, q_N\}$ with labels y_i, a policy π chooses lookups in sequence. With O_B^π, the set of fetched rows after B paid lookups, and M a ranking metric, the ultimate objective is $\max_\pi \mathbb{E}\left[M(f_\theta(Q \mid O_B^\pi), y)\right]$. However, ranking metrics are piecewise constant in the scores; thus, they don't reveal how much a single lookup helped. I therefore score a lookup a by its *batch value*, or the list-wide drop in cross-entropy ℓ:

$$V(a \mid O) = \sum_{i=1}^{N} \left[\ell\big(f_\theta(q_i \mid O), y_i\big) - \ell\big(f_\theta(q_i \mid O \cup \{a\}), y_i\big)\right]. \tag{4}$$

Summing cross-entropy over the list credits a shared entity for every prospect it informs. Importantly, this measures retrieval value, not decision value: lowering the loss of a prospect far from the cutoff raises V without necessarily changing who gets called. Section 4.3 looks into whether or not this matters. V needs labels so, for the purposes of the milestone, I use it only in a reference policy (oracle-greedy, Section 3.4); for my final submission, a second model will learn to predict it (H5).

3.4 Policies

Each policy listed here proposes the next lookups for a whole list in rounds of $N/4 = 32$ paid lookups. None of these five heuristic-policies are currently learned. **Random** picks uniformly among legal lookups; this acts as the floor policy. **Case-local** has the "least-researched" prospect take its own next lookup: its next table, then the entity its rows mention most. Tables are taken, deterministically, in alphabetical order; `event_attendees` is first on `rel-event`; Section 4.4 demonstrates that this ordering has consequences. **Uncertainty-first** executes lookups in a similar fashion with prospects ordered by $p(1-p)$, re-scored each round, as in per-instance active feature acquisition [4, 7]. **Coverage** is batch-aware: it makes the lookups that inform the most prospects first. An enriched entity informs every prospect whose pulled rows point to it, while a pull informs just one. **Sampled oracle-greedy** is a label-informed reference/oracle. Each round this policy samples 64 legal lookups, computes their true V with labels, and takes the best one. It is not deployable, and it serves as an approximate upper bound, though not a guaranteed one: it sees only 64 candidates per round and plans greedily.

3.5 Metrics

Experiment 1 reports RelBench's official validation AUROC on all 2,013 validation users, as mean $\pm$ s.d. over three seeds. Per call list, lookup experiments report AUROC, average precision (AP), and Precision@K: the share of true positives among the $K = \lceil 0.1N \rceil = 13$ highest-scored prospects. A random ranking scores the positive rate, about 0.11. I average over lists and give 95% percentile-bootstrap intervals over lists (2,000 resamples). Every policy runs on the same lists and subgraphs in a single run. Two limitations apply: the intervals cover list-to-list variation for one trained model on one day, not including training-seed variation; and since lists are drawn from the same 2,013 users (each user appears on about six of 100 lists), they are not fully independent, so the intervals are most likely narrow.

4 Results so far

All numbers below are for the `rel-event` validation day (2012-11-21) unless marked otherwise; `rel-f1` served as my development dataset. One NVIDIA L4 GPU was used for training and evaluation.

4.1 Validity checks

Implementation. Eleven unit tests pass, covering per-list metrics, temporal sampling, the environment's guarantees (full retrieval equals the full model, cache hits are free, pull/list before get/enrich), and ensuring that every policy spends exactly its budget in paid lookups using legal actions.

Temporal leakage. No sampled neighbor dated after its target's prediction time. However, target rows were flagged. RelBench's task table includes users before their recorded `joinedAt`: 35% of training targets and 52 of 2,013 validation targets. A target's own row is always in its subgraph; consequently, every model sees that `joinedAt`, LightGBM and RelBench's baselines included, and this leads to a temporal leak. I therefore removed `joinedAt` information from training and retrained every `rel-event` model. Thus, no model uses `users.joinedAt` as a feature. Notably, a user row's timestamp is also its `joinedAt`, so the GNN additionally must omit the age encoding ϕ of the prospect's own row; neighbors keep theirs, and the sampler's time filter is unchanged. All results in this report use the leakage-fixed models.

Why attendance history is not the label. Attendance rows are dated by their event's start time, so a prediction at t can only observe invitations to events that had started by t, while the label concerns invitations in the week after t (whose rows are never sampled). One assumption remains: a row's status is the database's final snapshot, so a late response to an earlier invitation would still be visible. Such responses are most likely rare but I have not measured this.

4.2 Training on partial views, and the decision gate (Experiment 1, Experiment 2)

Table 4 reports validation AUROC for when each distinct sampled row is revealed with probability p. The full-trained (all rows revealed) GNN scores 0.836 ± 0.019 with every row and 0.722 ± 0.018 with $p = 0$. The random-trained model's advantage is small: its mean is higher by 0.020 at $p = 0$ (0.742 vs. 0.722), and every other difference is at most 0.014 in either direction. With three seeds per regime, the difference at $p = 0$ is likely suggestive (Welch $t \approx 1.8$), and the rest are within seed variation. Precision@13 shows a larger gap with no rows revealed. From $p = 1$ to $p = 0$, the full-trained model drops from 0.586 to 0.189 and the random-trained model from 0.573 to 0.236.

Dataset	Training	$p = 1$	$p = 0.75$	$p = 0.5$	$p = 0.25$	$p = 0$
`rel-event`	full	0.836 ± 0.019	0.847 ± 0.015	0.859 ± 0.009	0.845 ± 0.007	0.722 ± 0.018
`rel-event`	random	0.845 ± 0.024	0.849 ± 0.024	0.845 ± 0.025	0.837 ± 0.016	0.742 ± 0.005
`rel-f1` (dev)	full	0.773 ± 0.027	0.778 ± 0.027	0.772 ± 0.028	0.763 ± 0.025	0.655 ± 0.031

LightGBM on the prospect's own row only, without `joinedAt`: `rel-event` 0.836 (0.853 with it), `rel-f1` 0.661

Table 4: Validation AUROC on all validation targets, mean $\pm$ s.d. over 3 training seeds, when each distinct sampled row is revealed with probability p (hidden rows dropped). Full: standard training. Random: a share $p \sim U(0, 1)$ revealed per training batch.

LightGBM baseline. LightGBM evaluated on a prospect's own row (locale, birthyear, gender, location, timezone) reaches 0.836 validation AUROC. This result ties the full-trained GNN with every row and is just below the random-trained GNN (0.845). However, on call lists it scores AP 0.495 and Precision@13 0.459, which lands well below the full-information GNN (about 0.60–0.62 and 0.58–0.60). One conclusion that can be drawn is that the trees use the profile row better: from that row alone, LightGBM scores 0.836 and the GNN with zero lookups 0.722–0.742. The GNN's advantage comes from the neighborhood, and it shows most with ranking the top of the list.

Experiment 2: the decision gate. Every policy lands between zero lookups and full information; if that gap were small, the policies would be nearly indistinguishable. Hence, I set a threshold before running frontiers: proceed only if $\Delta = \text{AUROC}(p{=}1) - \text{AUROC}(p{=}0) \geq 0.02$ for the full-trained model, since seed-to-seed spread on `rel-f1` was 0.02–0.05. On `rel-event`, $\Delta = 0.836 - 0.722 = 0.114$, so `rel-event` remains the dataset of record and `rel-avito` a stretch goal.

4.3 Budget frontiers (Experiment 3)

Figure 3 shows results from all five policies run on the same 100 total call lists, at budgets from $\frac{1}{4}$ to 16 paid lookups per prospect. The lists are scored by the seed-0 full-trained model, chosen before any

frontier runs. I summarize a policy at budget b via the share of the gap recovered,

$$\text{gap recovered}(b) = \frac{M_{\text{policy}}(b) - M_{\text{zero}}}{M_{\text{full}} - M_{\text{zero}}}, \tag{5}$$

where M_{zero} is a list's score with only the prospects' own rows and M_{full} is its score after full retrieval.

H1: concave frontiers. With just 32 paid lookups for 128 prospects ($b = \frac{1}{4}$, around 1.2% of the cost of full retrieval), case-local reaches 0.799 AUROC per list, against 0.701 with zero lookups and 0.819 with full information. That covers around $(0.799 - 0.701)/(0.819 - 0.701) = 83\%$ of the gap. From $b = \frac{1}{2}$, case-local exceeds full information (0.845 vs. 0.819), peaking at 0.863 at $b = 1$. On Precision@13 it reaches 0.522 at $b = 1$, against 0.569 with everything fetched. At $b = 16$, every heuristic except random is within 0.003 of full information. This is to be expected, as at larger budgets all policies have revealed nearly all of the same rows.

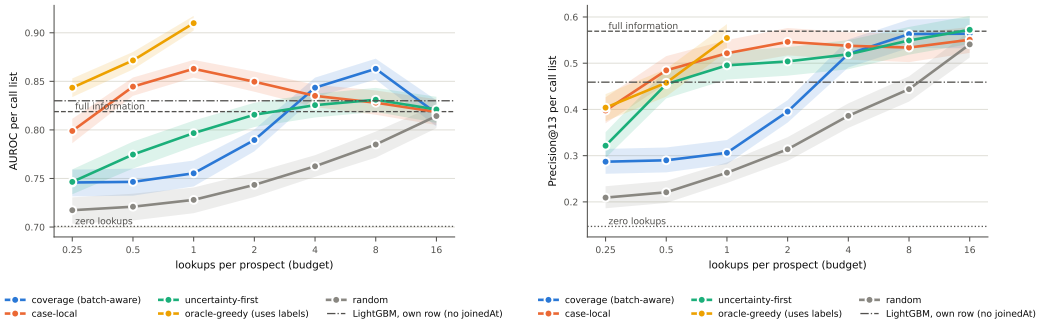
These zero and full-information results differ from Experiment 1's (0.722 and 0.836) for two reasons. First, experiment 3 uses only the seed-0 model, the weakest of the three (0.821 on the full validation set). And, second, it averages AUROC within 128-prospect lists instead of computing it once over all 2,013 users.

Paired differences with case-local exclude zero. Paired with case-local on the same lists at budget $b = 1$, the mean AUROC difference is -0.108 for coverage (95% interval $[-0.124, -0.091]$), -0.066 $[-0.077, -0.056]$ for uncertainty-first and -0.135 $[-0.150, -0.119]$ for random. Sampled oracle-greedy is $+0.047$ $[0.035, 0.059]$.

Why coverage loses early and wins later. The coverage policy maximizes *entity-level coverage*: how many prospects each lookup informs. However, a ranking also needs *prospect-level coverage*: how many prospects have any evidence at all. The two can pull in different directions, which can be illustrated as follows: in its first round, coverage has nothing listed yet, so it pulls 32 prospects. From then on, any event shared by two of those 32 informs two prospects, so it outranks every remaining pull, which informs just one. Here, I'll call this *starvation*. Case-local does the opposite: its first lookup for almost every prospect is the attendance table. Uncertainty-first starves in a different and more subtle way, returning to the same uncertain prospects each lookup. Later, once every prospect has evidence, shared entities become the better buy: coverage draws level with case-local at $b = 4$ ($+0.008$ $[-0.000, 0.018]$) and pulls ahead at $b = 8$ ($+0.035$ $[0.029, 0.040]$). Together these results suggest a two-stage strategy: first give every prospect a minimum of evidence, then spend on shared entities. That is the breadth-then-share heuristic I test next (Section 5).

Ranking quality is not decision quality. Oracle-greedy picks lookups by their true V and wins on both metrics at $b = 1$: $+0.047$ AUROC, and Precision@13 0.555 vs. 0.522 (paired $+0.033$ $[0.005, 0.061]$). Ranking quality and the top-13 decision still diverge elsewhere though. Case-local at $b = 1$ beats full information on AUROC (0.863 vs. 0.819) but trails it on Precision@13 (0.522 vs. 0.569; paired -0.048 $[-0.061, -0.035]$). With the random-trained model, coverage beats case-local on AUROC at $b = \frac{1}{4}$ but loses on Precision@13 (below). A better ranking overall does not necessarily put the right prospects in the top 13. Whether a cutoff-aware value beats V, therefore, remains an open question for H5.

H2 under chosen lookups. Figure 4 repeats the frontier with the seed-0 random-trained model on 30 lists. The random-trained model starts higher with zero lookups, 0.763 against 0.701. With case-local it reaches 0.860 at $b = 4$, above its own full information (0.852) and above the LightGBM reference (0.830), though the full-trained model's case-local peak in Figure 3 is similar (0.863 at $b = 1$). With this model, policy performance order changes at small budgets. Uncertainty-first and coverage beat case-local at $b = \frac{1}{4}$ (paired $+0.023$ $[0.005, 0.040]$ and $+0.019$ $[0.010, 0.027]$), and uncertainty-first also leads on Precision@13 (0.418 vs. 0.385; paired interval $[-0.000, 0.067]$). Coverage does not: it trails case-local on Precision@13 (0.333 vs. 0.385; paired -0.051 $[-0.090, -0.013]$), so its AUROC gain does not reach the top $K = 13$. Case-local catches up in performance once every prospect's attendance has been pulled ($b = 1$: 0.850). Under these conditions, uncertainty became a useful guide only when the model had been trained on partial views: with the full-trained model, uncertainty-first trails case-local at $b = \frac{1}{4}$ (-0.052 $[-0.065, -0.041]$).



(a) AUROC per call list (b) Precision@13 per call list

Figure 3: Budget frontiers on `rel-event`: mean metric per 128-prospect call list against paid lookups per prospect (log scale), 100 lists, seed-0 full-trained model; bands are 95% bootstrap intervals over lists. Dotted: zero lookups. Dashed: full information (on average 2,752 paid lookups per list). Dash-dot: LightGBM on the prospect's own row without `joinedAt`, scored on 200 random lists of the same day. Oracle-greedy is the sampled, label-informed reference policy and is run only to $b = 1$.

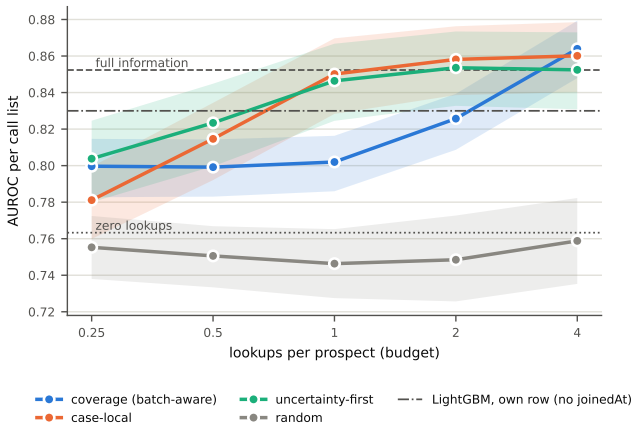


Figure 4: The AUROC frontier with the random-trained model (seed 0): 30 call lists, four policies, 95% bootstrap bands. Dotted: zero lookups (0.763). Dashed: full information (0.852). Dash-dot: LightGBM on the prospect's own row without `joinedAt` (0.830). A separate run from Figure 3, so the two are not paired list by list.

4.4 What is actually worth a lookup? (Experiment 4)

Table 5 pulls one table for every prospect, then enriches everything those rows list, measuring the AUROC gain over zero lookups. Under the current model and training regime, one table carries most of the retrieval value. Pulling attendance histories (89 paid lookups per list) raises AUROC by 0.153 $[0.123, 0.180]$, and enriching the roughly 1,000 events they list does not add anything more. Interest clicks and the events a prospect created change very little, with intervals including zero. Pulling friendships raises AUROC by 0.034 $[0.010, 0.057]$, but also enriching the friends those rows list converts a loss of 0.054.

Attendance rows hold each past invitation's response (invited, yes, maybe, no), the most direct evidence of whether a user answers invitations; Section 4.1 explains why they do not reveal the future label. Friends behave differently: the friendship rows themselves help, but enriching the friends they list hurts. This suggests the model, trained with every row present, does not use friends' own rows well on this day, either because (speculatively) they add noise or because the many enriched friend rows swamp the summed message at layer 2. Separating these needs the ablations described in Section 5.

The results in the table also explain two results from Experiment 3. First, case-local leads at small budgets likely because its first lookup for each prospect is an attendance pull, the most valuable lookup available (+0.153 AUROC for 89 lookups). Second, case-local can beat full information

(0.863 vs. 0.819 at $b = 1$) because full retrieval also enriches friends, which lowers AUROC by 0.054. Notably, case-local reaches attendance first only because it takes tables in alphabetical order and `event_attendees` sorts first, so its lead reflects a lucky match between lookup order and where the value sits. A case-local with a value-aware or randomized table order would serve as a more fair comparison (Section 5).

Table pulled for every prospect	Pull only			Pull + enrich	
	lookups	AUROC gain [95% interval]	per 100	lookups	gain
`event_attendees`	89	$+0.153$ $[+0.123, +0.180]$	$+0.171$	1,119	$+0.152$
`event_interest`	11	-0.016 $[-0.034, +0.002]$	-0.147	93	-0.018
`events` (created)	7	-0.008 $[-0.024, +0.007]$	-0.111	183	-0.014
`user_friends`	128	$+0.034$ $[+0.010, +0.057]$	$+0.026$	1,448	-0.054

Table 5: Value of each table: AUROC gain over zero lookups when one table is pulled for every prospect (and, right, when everything those rows list is enriched too), mean over 30 call lists, seed-0 full-trained model (`scripts/table_value.py`). "Per 100": gain per 100 paid lookups. Before the `joinedAt` fix, the same table gave attendance $+0.224$, created events -0.029 and friendships -0.062.

4.5 Sharing and batch-awareness (Experiment 5)

For this experiment, I split 100 call lists into thirds by sharing (Eq. 1, over all entity types) and compare coverage with case-local, averaging the paired AUROC difference over $b \leq 2$. Coverage trails case-local on every third: -0.079 $[-0.100, -0.060]$ on the least-shared third (11–17% shared), -0.070 $[-0.085, -0.055]$ on the middle third (17–19%), and -0.090 $[-0.109, -0.071]$ on the most-shared third (19–24%). The most-shared third shows the largest deficit, but the intervals overlap, and the per-list correlation between sharing and coverage's advantage is around zero (0.02). The mechanism matches Experiment 3: the coverage policy budget goes to events shared among the prospects it pulled first, and other prospects are left without evidence. At small budgets, then, coverage contradicts H4, though this reflectsan issue of starvation rather than batch-aware retrieval in general. The test is also limited: sharing ranges only from 11% to 24% across random lists of one day.

Summary against the hypotheses. H1: supported (83% of the gap at about 1% of the cost; 81% on a separate 30-list run before the fix). H2: only weakly supported: under random revelation the random-trained model leads only with nothing revealed (by 0.020, $t \approx 1.8$), and under chosen lookups it starts higher (0.763 vs. 0.701) but peaks about where the full-trained model does (0.860 vs. 0.863). H3 (re-scoped): supported for this model (attendance histories carry most of the retrieval value; friendship pulls add a little, and enriching friends lowers AUROC). H4: not supported at small budgets by the one batch-aware heuristic tested, which starves prospects at every level of sharing; a batch-aware advantage appears at $b = 8$ and is borderline at $b = 4$.

5 Next steps

My milestone results change what the learned policy must do. It first has to cover every prospect, know which table is worth a lookup, and value moving prospects across the call cutoff rather than only lowering loss. Table 6 orders the remaining work by priority.

The learned policy (H5). H5 predicts that a learned lookup value will beat the best heuristic. Since ranking quality and the top-13 decision can diverge (Experiment 3), I will attempt to learn two targets: the batch value V, and a cutoff-aware variant that weights each prospect by its closeness to the top-13 cut-off boundary. On labeled training-day lists, I will run existing policies to different budgets and compute both values for sampled candidate lookups. A model will then learn to predict them without labels, first from hand-built features, then with a GNN on the revealed graph [10]. At test time, the policy repeatedly takes the highest-valued lookup and re-scores, and will then be evaluated in the same frontier runner as the heuristics.

	Experiment	Why
1	Breadth-then-share heuristic	Fixes starvation; tests the two-stage strategy
2	Case-local with value-aware and randomized table order	Tests whether lookup order explains its lead
3	Learned, cutoff-aware lookup value (H5)	Aligns the policy with the top-13 decision
4	Full- vs. random-trained frontiers, paired, more seeds	Isolates the effect of partial-view training
5	Mask vs. drop mode	Tests representing unfetched rows
6	Mean aggregation, GAT [8], LightGBM score as a GNN input	Do the findings depend on the GNN?
7	Larger lists ($N \in \{32, 128, 512\}$), test day, `rel-avito`	External validity

Table 6: Remaining work (in order of priority).

References

[1] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[2] Matthias Fey, Weihua Hu, Kexin Huang, Jan Eric Lenssen, Rishabh Ranjan, Joshua Robinson, Rex Ying, Jiaxuan You, and Jure Leskovec. Position: Relational deep learning – graph representation learning on relational databases. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 13592–13607, 2024.

[3] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems 30*, 2017.

[4] Alexander Luke Ian Norcliffe, Changhee Lee, Fergus Imrie, Mihaela van der Schaar, and Pietro Liò. Stochastic encodings for active feature acquisition. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 46784–46814, 2025.

[5] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.

[6] Joshua Robinson, Rishabh Ranjan, Weihua Hu, Kexin Huang, Jiaqi Han, Alejandro Dobles, Matthias Fey, Jan E. Lenssen, Yiwen Yuan, Zecheng Zhang, Xinwei He, and Jure Leskovec. RelBench: A benchmark for deep learning on relational databases. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024) Datasets and Benchmarks Track*, 2024.

[7] Michael Valancius, Maxwell Lennon, and Junier Oliva. Acquisition conditioned oracle for nongreedy active feature acquisition. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 48957–48975, 2024.

[8] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

[9] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019.

[10] Zihe Zhou. Budgeted task-aware acquisition of dynamic networks. *arXiv preprint arXiv:2609.05862*, 2026.